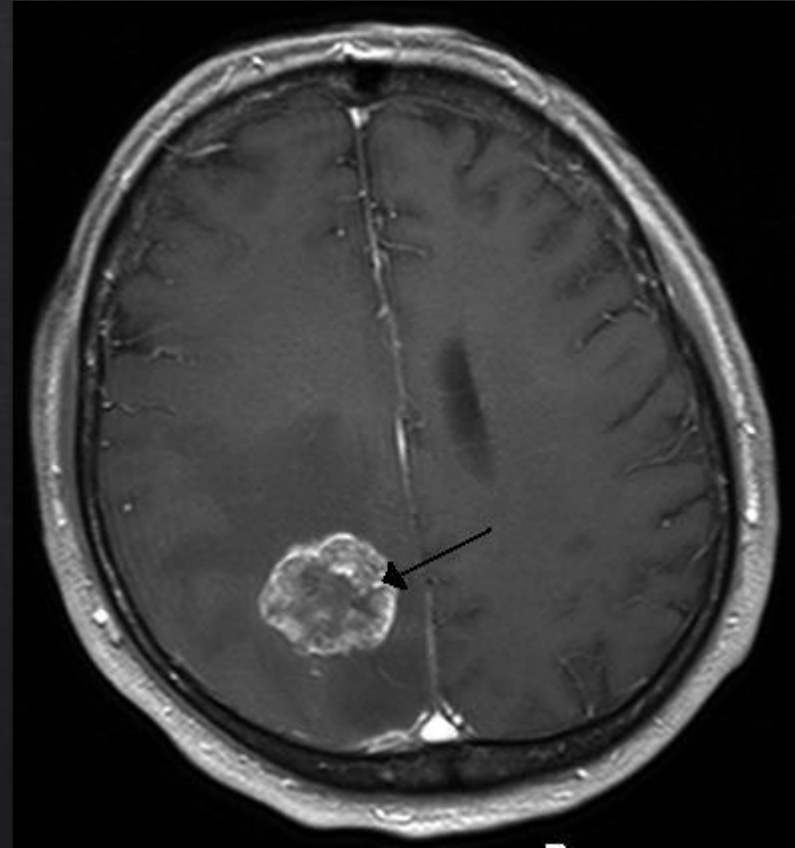


Brain Tumour Detection

By Amal, Devadath, Vighnesh

PROBLEM STATEMENT

Manual diagnosis of brain tumors from medical images (primarily **Magnetic Resonance Imaging (MRI)** scans) by radiologists is essential but presents a significant challenge it is **Time-Consuming & Labor-Intensive** Analyzing hundreds of 2D slices in a typical MRI scan is a time-intensive process that can delay critical treatment.



Data Set Split

Set	Size (Samples)	Percentage (%)	Purpose in ML Pipeline
Train Set	5,712	83.6%	Used to train the model (adjust its internal parameters/weights).
Validation Set	667	9.8%	Used to tune hyperparameters and evaluate the model <i>during</i> training to prevent overfitting.
Test Set	644	9.4%	Used for the final, unbiased evaluation of the chosen model's performance.
Total Dataset Size	7,023	100%	

CNN Model

- ◆ CNN with 3 of each (convolution, relu and max pooling layers) and fully connected layer (w/ 128 neurons, drops out 40% of neurons at random, sigmoid activation)
- ◆ Adam Optimizer is used
- ◆ Trained for 100 epochs with early stopping enabled with patience of 15
- ◆ Learning rate : 0.0001

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DeeperCNN(nn.Module):
    def __init__(self):
        super(DeeperCNN, self).__init__()
        self.cnn_model = nn.Sequential(
            nn.Conv2d(3, 16, 5), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(16, 32, 5), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, 5), nn.ReLU(), nn.MaxPool2d(2, 2)
        )
        self.fc_model = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 12 * 12, 128),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(128, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = self.cnn_model(x)
        x = self.fc_model(x)
        return x
```


Overfitting Prevention

- ◆ Augmentation – random flips and rotates
- ◆ Update with best model – best validation set accuracy
- ◆ Early stopping
- ◆ Dropout
- ◆ Batch Training

```
def augment(self, img):  
    if random.random() > 0.5:  
        img = cv2.flip(img, 1)  
    if random.random() > 0.5:  
        angle = random.randint(-15, 15)  
        h, w = img.shape[:2]  
        M = cv2.getRotationMatrix2D((w/2, h/2), angle, 1)  
        img = cv2.warpAffine(img, M, (w, h))  
    return img
```

Model Accuracy

Accuracy: 99.38%

Classification Report:

	precision	recall	f1-score	support
0.0	0.98	1.00	0.99	199
1.0	1.00	0.99	1.00	445

Confusion Matrix:

```
[[199  0]
 [  4 441]]
```

Code Snippets

```
class MRI(Dataset):
    def __init__(self, mode='train'):
        if mode == 'train':
            tumour_path = "data/train/tumor/*.jpg"
            no_tumour_path = "data/train/no_tumor/*.jpg"
        elif mode == 'val':
            tumour_path = "data/val/tumor/*.jpg"
            no_tumour_path = "data/val/no_tumor/*.jpg"
        else:
            tumour_path = "data/test/tumor/*.jpg"
            no_tumour_path = "data/test/no_tumor/*.jpg"

        self.data = glob.glob(tumour_path) + glob.glob(no_tumour_path)
        self.labels = [1]*len(glob.glob(tumour_path)) + [0]*len(glob.glob(no_tumour_path))

    def __len__(self):
        return len(self.data)

    def augment(self, img):
        if random.random() > 0.5:
            img = cv2.flip(img, 1)
        if random.random() > 0.5:
            angle = random.randint(-15, 15)
            h, w = img.shape[:2]
            M = cv2.getRotationMatrix2D((w/2, h/2), angle, 1)
            img = cv2.warpAffine(img, M, (w, h))
        return img

    def __getitem__(self, idx):
        img_path = self.data[idx]
        label = self.labels[idx]
        img = cv2.imread(img_path)
        img = cv2.resize(img, (128, 128))
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        if "train" in img_path:
            img = self.augment(img)
        img = img / 255.0
        img = np.transpose(img, (2, 0, 1))
        return torch.tensor(img, dtype=torch.float32), torch.tensor(label, dtype=torch.float32).unsqueeze(0)
```

```
def train_system():
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    train_loader = DataLoader(MRI('train'), batch_size=BATCH_SIZE, shuffle=True)
    val_loader = DataLoader(MRI('val'), batch_size=BATCH_SIZE, shuffle=False)
    model = DeeperCNN().to(device)
    criterion = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=LR)
    best_val_loss = float('inf')
    patience_counter = 0

    for epoch in range(EPOCH):
        model.train()
        train_loss = 0
        for X, y in train_loader:
            X, y = X.to(device), y.to(device)
            optimizer.zero_grad()
            loss = criterion(model(X), y)
            loss.backward()
            optimizer.step()
            train_loss += loss.item()

        model.eval()
        val_loss = 0
        with torch.no_grad():
            for X, y in val_loader:
                X, y = X.to(device), y.to(device)
                val_loss += criterion(model(X), y).item()

        avg_train = train_loss / len(train_loader)
        avg_val = val_loss / len(val_loader)
        print(f"Epoch {epoch+1} Train Loss: {avg_train:.4f} | Val Loss: {avg_val:.4f}")

        if avg_val < best_val_loss:
            best_val_loss = avg_val
            torch.save(model.state_dict(), "best_mri_model_v2.pth")
            patience_counter = 0
        else:
            patience_counter += 1

    if patience_counter > PATIENCE:
        break
```

THANK YOU